สรุปการเชื่อมต่อ API ภายนอก

ระบบร้านอาหารออนไลน์ "อร่อยเกินเบอร์"

ไฟล์รวมฟังก์ชัน API ภายนอก	myapp/services.py
จำนวนบริการที่เชื่อมต่อ	5 บริการ
แบบใช้ API Key	Line Notify (Token), Google OAuth (Client ID/Secret), SMTP (ตาม backend)
แบบไม่ใช้ Key (ฟรี)	Google Maps Embed, Nominatim/OpenStreetMap
ไลบรารี	requests (ส่งคำขอ) | reportlab (สร้างเอกสารฉบับนี้)
การทดสอบ	mock ทั้งหมดใน myapp/tests.py — 178 เทสต์

หลักการออกแบบการเชื่อมต่อ API ภายนอก

แยกงานภายนอกไว้ในไฟล์เดียว `myapp/services.py`	หน้าเว็บ/API ภายในเรียกผ่านฟังก์ชันพวกนี้ ไม่เรียก requests เอง
เก็บความลับไว้นอกโค้ด	token/key ผ่านตัวแปรสภาพแวดล้อม (env) หรือฐานข้อมูล (StoreLocation) ไม่ฝังใน source
กันระบบพัง	ทุกฟังก์ชันจับ error แล้วคืน False/None (เช่น เน็ตหลุดแค่ข้ามการแจ้งเตือน ไม่ทำให้สั่งอาหารผิด)
หมดเวลาทุกครั้ง	ทุกการเรียกใช้ timeout=8 วินาที ไม่ปล่อยให้ค้างคา request ไว้
เทสต์โดยไม่ยิง API จริง	ใช้ unittest.mock แทน requests.get/post ทั้งหมดใน myapp/tests.py (178 เทสต์ผ่าน)
ตั้งค่า/จัดการได้จากหน้าเว็บ	ปุ่ม "ค้นหาพิกัดที่อยู่" (Geocode), "ส่งข้อความทดสอบ" (Line), ตั้งตำแหน่งร้านในหน้าตั้งค่าเดลิเวอรี่

ตารางรวมบริการภายนอกทั้งหมด

บริการ	ใช้ทำอะไร	ต้องใช้ Key?
Google Maps Embed	แสดงแผนที่ตำแหน่งร้าน / ปลายทางเดลิเวอรี่บนหน้าเว็บ (iframe)	ไม่ต้องใช้ API Key (ฟรี)
Line Notify	ส่งแจ้งเตือนออเดอร์ใหม่ / จองโต๊ะ / รีวิวใหม่ ไปยัง Line ของร้าน	Line Notify Token — ตั้งที่ Admin (StoreLocation.line_notify_token) หรือตัวแปรสภาพแวดล้อม LINE_NOTIFY_TOKEN
Nominatim / OpenStreetMap	เปลี่ยนที่อยู่จัดส่ง (ข้อความ) → พิกัด GPS แล้วคำนวณระยะทาง/ค่าส่งอัตโนมัติ	ไม่ต้องใช้ API Key — มีข้อจำกัด 1 คำขอ/วินาที ใช้เฉพาะตอนกดปุ่ม "ค้นหาพิกัดที่อยู่"
Google OAuth 2.0 (Sign-In)	ให้ผู้ใช้ล็อกอินด้วยบัญชี Google แทนการสมัครสมาชิกแบบฟอร์ม	GOOGLE_OAUTH_CLIENT_ID + GOOGLE_OAUTH_CLIENT_SECRET (env) + SocialApp ในฐานข้อมูล
Email (SMTP)	ส่งอีเมลแจ้งเตือน + ลิงก์รีเซ็ตรหัสผ่าน	คอนฟิกผ่าน env: DJANGO_EMAIL_BACKEND, DEFAULT_FROM_EMAIL (และ SMTP_HOST/USER/PASSWORD ตาม backend)

1. Google Maps Embed — แสดงผล (Display) — โหลดจากฝั่งเบราว์เซอร์

Endpoint: https://maps.google.com/maps?output=embed

วัตถุประสงค์	แสดงแผนที่ตำแหน่งร้าน / ปลายทางเดลิเวอรี่บนหน้าเว็บ (iframe)
วิธีเรียกใช้	แทรก <iframe src="https://maps.google.com/maps?q=<lat>,<lng>&z=<zoom>&output=embed">
การยืนยันตัวตน	ไม่ต้องใช้ API Key (ฟรี)
ไฟล์ที่เกี่ยวข้อง	myapp/templates/core/contact.html, myapp/templates/delivery/delivery_settings.html, myapp/templates/order/order_detail.html
หมายเหตุ	เลือกตำแหน่งใหม่ ใช้ปุ่ม "เปิด Google Maps เต็มจอ" จากนั้นก๊อปปี้พิกัดมาวางที่ช่องละติจูด/ลองจิจูด (แผนที่ฝั่งเว็บจะอัปเดตให้อัตโนมัติผ่าน JS)

2. Line Notify — ส่งออก (Outbound) — ฝั่งเซิร์ฟเวอร์

Endpoint: POST https://notify-api.line.me/api/notify

วัตถุประสงค์	ส่งแจ้งเตือนออเดอร์ใหม่ / จองโต๊ะ / รีวิวใหม่ ไปยัง Line ของร้าน
วิธีเรียกใช้	requests.post() Header: Authorization: Bearer ตัวแปร: message
การยืนยันตัวตน	Line Notify Token — ตั้งที่ Admin (StoreLocation.line_notify_token) หรือตัวแปรสภาพแวดล้อม LINE_NOTIFY_TOKEN
ไฟล์ที่เกี่ยวข้อง	myapp/services.py: notify_line(), line_notify_status(), announce_*(); myapp/views.py: line_notify_test; URL: /delivery/settings/test-line/
หมายเหตุ	หมดเวลา 8 วินาที/ครั้ง, ลองใหม่ (Retry) รวม 3 ครั้งเมื่อเจอ 429/5xx/เน็ตหลุด — token ผิด (400) จบเลย ไม่เสียเวลาลองซ้ำ; มีปุ่ม "ส่งข้อความทดสอบ" ที่หน้าตั้งค่าเดลิเวอรี่; token แสดงแบบซ่อนใน Admin

3. Nominatim / OpenStreetMap — ส่งออก (Outbound) — ฝั่งเซิร์ฟเวอร์

Endpoint: GET https://nominatim.openstreetmap.org/search

วัตถุประสงค์	เปลี่ยนที่อยู่จัดส่ง (ข้อความ) → พิกัด GPS แล้วคำนวณระยะทาง/ค่าส่งอัตโนมัติ
วิธีเรียกใช้	requests.get() ตัวแปร: q=, format=json, limit=1, countrycodes=th Header ต้องมี User-Agent ตามนโยบาย OSM
การยืนยันตัวตน	ไม่ต้องใช้ API Key — มีข้อจำกัด 1 คำขอ/วินาที ใช้เฉพาะตอนกดปุ่ม "ค้นหาพิกัดที่อยู่"
ไฟล์ที่เกี่ยวข้อง	myapp/services.py: geocode_address(); myapp/models.py: Delivery.geocode_from_address(); myapp/views.py: delivery_geocode; URL: /delivery//geocode/
หมายเหตุ	จับ error ทั้งหมดคืน None (ไม่ทำให้สั่งอาหารพัง); เจอพิกัดแล้ว Delivery.save() คำนวณระยะทาง (Haversine) + ค่าส่งให้เอง

4. Google OAuth 2.0 (Sign-In) — OAuth (Web Redirect flow) — ฝั่งเบราว์เซอร์ + เซิร์ฟเวอร์

Endpoint: ระบบ OAuth ของ Google (ผ่าน django-allauth)

วัตถุประสงค์	ให้ผู้ใช้ล็อกอินด้วยบัญชี Google แทนการสมัครสมาชิกแบบฟอร์ม

วิธีเรียกใช้	allauth.socialaccount.providers.google — เบราว์เซอร์พาผู้ใช้ไปหน้า Google แล้วรับ token กลับมา
การยืนยันตัวตน	GOOGLE_OAUTH_CLIENT_ID + GOOGLE_OAUTH_CLIENT_SECRET (env) + SocialApp ในฐานข้อมูล
ไฟล์ที่เกี่ยวข้อง	myproject/settings.py; myapp/views.py: google_login_redirect; myapp/management/commands/create_google_app.py
หมายเหตุ	ตั้งค่าได้ผ่านคำสั่ง `python manage.py create_google_app` — เลือนเปิด Social login เมื่อข้อมูลครบ แล้วหน้า Login จะแสดงปุ่ม "เข้าสู่ระบบด้วย Google"

5. Email (SMTP) — ส่งออก (Outbound) — ฝั่งเซิร์ฟเวอร์

Endpoint: โปรโตคอล SMTP (ตาม EMAIL_BACKEND)

วัตถุประสงค์	ส่งอีเมลแจ้งเตือน + ลิงก์รีเซ็ตรหัสผ่าน
วิธีเรียกใช้	Django send_mail() — โหมดพัฒนา (default) ใช้ Console backend ส่งอีเมลลง terminal เพื่อไม่ต้องตั้ง SMTP จริง
การยืนยันตัวตน	คอนฟิกผ่าน env: DJANGO_EMAIL_BACKEND, DEFAULT_FROM_EMAIL (และ SMTP_HOST/USER/PASSWORD ตาม backend)
ไฟล์ที่เกี่ยวข้อง	myapp/services.py: notify_email(); myapp/templates/core/auth/password_reset_form.html (เจเนอเรต URL ยืนยันจาก Django)
หมายเหตุ	notify_email() ยอมรับ list/str และพยายามส่งแบบ fail_silently เพื่อไม่ให้บางไปเพราะอีเมล

หมายเหตุด้านความปลอดภัย

• Line Notify token ถูกซ่อนผ่าน mask_token() — Admin โชว์แค่ท้าย 4 ตัว, REST API /api/store/ กรองข้อมูลนี้ไม่ให้รั่วออกนอก

• Nominatim ต้องส่ง User-Agent บอกตัวตนทุกครั้ง ตามนโยบายการใช้งานของ OpenStreetMap (GeoNames/OSMF policy)

• Google OAuth ใช้ SOCIALACCOUNT_ONLY + ตรวจ EMAIL_VERIFICATION='none' เพื่อให้เข้าสู่ระบบรวดเร็วแต่ยืนยันตัวตนจาก Google แล้ว

• ฟีเจอร์ลากหมุดเพื่อตั้งพิกัด ต้องใช้ Google Maps JavaScript API (ต้องมี API Key) — โครงการนี้ใช้วิธีกรอกพิกัด + ลิงก์เปิด Google Maps แทน (embed แบบไม่ใช้ Key)